

★第4章

重圧から生み出される小手先の指標

《登場人物》

湊太郎

山田健一

佐藤

飛鳥さくら

高橋美咲

4-1

小手先の指標に依存せずに 問題を特定する手法

ストーリー

ワークショップの準備

金曜日の午後、大会議室に湊は一人で準備をしていた。窓から差し込む光がホワイトボードを照らし、湊は付箋を並べながら緊張を感じている。

先週、一人でデータ分析をしていた湊は、山田からの後押しを受けて、チーム全体で問題を共有するワークショップを企画した。忙しいなか時間を割いてもらう中で、もし誰も本音を言わなかったらこのワークショップは意味のないものになってしまう。

湊は付箋を数えながら考えていた。リーダーとして、チームを正しい方向に導く責任がある。でも、その方法がわからない。一人でデータ分析をしていた時は数字を見つめていれば良かったが、今は違う。人を動かし、本音を引き出し、共通の課題を見つけなければならない。

「準備手伝うよ」

背後から声がかかった。振り返ると、山田がドア際に立っていた。

「山田さん、ありがとうございます」

「湊くんが一人で抱え込まないで、チーム全体で考えようって決めたのは正解だと思うよ。僕も昔、同じような会を開催したことがあるんだ」

山田はホワイトボードにマーカーを並べ、少し間を置いてから口を開いた。

「でも、最初はみんな遠慮がちかもしれない。特に、別組織の飛鳥や高橋さんは、開発チームの内情を直接言いにくいかもしれない。別組織だから、遠慮してしまうんだよね」

「そうですね……どうやって本音を引き出せばいいのでしょうか」

「まずは、湊くんが率先して具体的な痛みを話すことだね。リーダーが本音を言えば、他の人も話しやすくなる。それから、誰の発言も**否定しないこと**。『それは違う』とか『それは問題じゃない』とか言わない。どんな小さな痛みでも、まずは受け止めることが大事だよ」

山田はマーカーを置き、湊の方を向く。

「あと、飛鳥と高橋さんには、最初に『別組織だけど、今日は開発チームの一員として意見を聞かせてほしい』って伝えるといいかもしれない。そうすれば、遠慮せずに話してくれるはずだ」

「なるほど……ありがとうございます」

湊は山田のアドバイスを頭の中で整理していた。リーダーとして率先して本音を言う。誰の発言も否定しない。別組織のメンバーにも配慮する。これらを意識すれば、きっと本音を引き出せるはずだ。

時計を見ると、開始時刻まであと10分。

メンバーが集まる

開始時刻が近づくにつれ、佐藤、他2名のチームメンバー、飛鳥、高橋と順に揃い、全員が会議室に集まった。飛鳥が「今日はどんな内容ですか？」と聞き、高橋が「すみません、少し遅れました」と席につく。湊は深呼吸をして、全員に向き直った。

「今日はお忙しい中、ありがとうございます。先週、技術的負債の影響を可視化し

たんですが、それだけでは不十分だと思って。みんなで日々感じている『痛み』を共有して、一緒に改善策を考えたいんです」

湊は山田のアドバイスを思い出しながら続ける。

「飛鳥さん、高橋さん、今日は別組織から来ていただいておりますが、今日は開発チームの一員として、遠慮せずに意見を聞かせてください。開発チームだけでは見えない視点を、ぜひ教えてほしいんです」

ぎこちない開始

湊がワークショップの説明を終えると、会議室に沈黙が訪れた。最初の5分間、誰もが遠慮がちに付箋を見つめている。手元のパソコンで何を確認する人、ただただ空気を見つめる人。それぞれが何かを言おうとしているが、誰も最初の一步を踏み出せない。エアコンの音だけが低く響く。

湊は山田の言葉を思い出していた。でも、何を話せばいいのか。湊自身も、まだ整理できていない痛みがたくさんあり、口を開くきっかけが掴めない。

「まず、日々の開発で感じている『痛み』を付箋に書いてみませんか？ どんな小さなことでも構いません。例えば、『今日も残業になった』とか、『また仕様が変わった』とか、そんなことでもいいんです」

湊の言葉に、メンバーたちは遠慮がちに書き始めた。最初は当たり障りのない内容ばかりだった。

佐藤が小さく声を出して読む。

「時間が足りない」

別のメンバーが続ける。

「仕様変更が多い」

「残業が多い」

付箋がホワイトボードに貼られていく。でも、まだ表面的な内容ばかりだ。湊は内心で焦っていた。このままでは、本当の問題が見えてこない。

山田の勇気ある発言

その時、山田が立ち上がった。椅子を押し、ホワイトボードの前に歩み寄ってマーカーを手取る。

「じゃあ、僕から具体的な話をしますね。技術的負債で新機能が作りにくい。同じ規模の機能開発に、以前は3日かかっていたのが、今は1週間かかる。コードが複雑になりすぎて、どこを触っても影響範囲が広がる。テストを追加するのも大変だし、リファクタリングする時間もない」

山田は付箋に書きながら続ける。ペンの音だけが会議室に響く。

「特に問題なのは、スピードを優先してテストを書かないことだ。テストがないと、変更の影響範囲がわからない。一箇所を修正したら、どこまで影響するか予測できない。だから、予想外の場所でエラーが出る」

「それから、うちのシステム、最初は綺麗に分かれてたんだよ。勤怠は勤怠、経費は経費って。でも3年間で機能を足し続けた結果、どこがどこと繋がってるのか誰もわからなくなった。設計図を書き直す時間を取らなかったツケだ」

「それから、バグ修正のたびに別の場所が壊れる。モグラ叩きみたいな状態が続いている。先週も、経費精算のバグを修正したら、勤怠管理の画面が表示されなくなった。なんでそんなことが起きるのか、自分でもわからない。これが、僕が感じている痛みです」

山田が付箋をホワイトボードに貼った瞬間、会議室の空気が少し変わった。佐藤が小さく「わかる……」と呟く。別のメンバーも深く頷いている。山田の具体的な発言が、他のメンバーにも勇気を与えたようだ。少しずつ、本音を出し始めた。

本音の爆発

山田の発言の後、再び沈黙が訪れた。でも、今度は違う種類の沈黙だった。誰かが何かを言おうとしている。そんな緊張感が会議室に漂っている。

高橋が小さく息を吸い込む。両手を膝の上に置き、背筋を伸ばしてから、勇気を振り絞るようにはっきりと言った。

「正直、テスト時間が全然足りません。仕様変更が週に3-5回で、テストケースも毎回見直さないといけない。でもリリース日は変わらないから検証時間がどんどん削られていく」

高橋の声は少し震えていたが、その言葉には強い意志が込められていた。

「先月のリリースでも、最終テストでバグが3件見つかったんです。でも、リリース日が決まっているから、そのままリリースしてしまった。結果として、リリース後にユーザーからエラー報告が殺到して、徹夜で対応することになりました。QA部として、品質に責任を持っているんですが、テスト時間がスケジュールに入らないことが多いんです。開発チームには悪いと思いつつも、もっと時間をかけてほしいって思ってしまう」

高橋の言葉に、エンジニアのメンバーたちも頷く。佐藤が「すみません……」と小さく謝る。別のメンバーも「確かにテスト時間を削ってしまっていて……」と反省の表情を見せた。誰もが同じ経験をしていた。でも、それを口に出すことができなかった。

高橋の言葉がきっかけで、堰を切ったように本音が溢れ出した。

「コードレビューする時間がない。急いでコードを書いて、レビューを依頼するけど、レビューする側も忙しくて時間が取れない。結局、レビューなしでマージしてしまうこともある。それで後からバグが見つかって……」

佐藤が小さく呟く。

「見積もりがいつも外れる理由がわからない。失敗パターン『とりあえず作ってから考える』に陥ってる。最初は『3日でできる』って言ったのに、実際には1週間かかってしまう。その理由を説明するのが難しい」

「プロジェクト管理モジュールも同じ。データベースの構造が複雑で、新しい機能を追加するのが大変。同じようなコードが3箇所に散らばっていて、全部修正しないと動かない。でも、全部修正するのは時間がかかりすぎる」

別のメンバーが続ける。

「勤怠管理モジュールのコード、前の担当者が書いたんだけど、なぜこの設計にしたのかわからない。その人に聞かないと理解できない。でも、その人はもう別のプロジェクトに移ってしまっ……」

「ライブラリのバージョンも古いまだ。セキュリティパッチを適用したいけど、バージョンアップすると動かなくなる可能性がある。でも、このまま放置するのもリスクだし……」

PMの飛鳥が手を挙げた。

「プロダクト部としても、悩みがあるんです。ユーザーや経営層から機能追加の要望が来る。それを開発チームに伝えると、『技術的に難しい』とか『時間がかかる』とか言われる。でも、ビジネス的には早く出したい。そのバランスを取るのが難しいんです」

飛鳥が静かに付箋を貼った。湊がそれを読むと、こう書いてあった。「スピードを優先して、開発チームに無理をさせてしまっている。でも止められない」。PMらしい勢いのある飛鳥とは違う、小さな文字だった。

付箋がどんどん増えていく。ホワイトボードに貼られた付箋は、すでに30枚を超えていた。そして、まだ書き足りない様子のメンバーもいる。

湊は付箋を見つめながら、改めて問題の深刻さを実感していた。一人で抱え込んでいた問題が、実はチーム全体の課題だった。一瞬、時間が止まったような気

がした。エンジニアだけの問題ではなく、QA 部、プロダクト部、それぞれが同じような悩みを抱えていた。言葉にできなかったものが、一気に表に出た手応えがある。

「みんな、同じようなことで悩んでたんですね……」

湊は小さく呟く。その言葉に、全員が深く頷く。

その夜、湊は帰宅して窓際のパキラに目をやった。幹の根元に、小さな新芽が出ていた。

痛みのグルーピング

付箋が 30 枚を超えたところで、湊は次のステップを提案した。

「じゃあ、これらの痛みを整理してみましょう。似た内容をグループに分けてみませんか？ そうすれば、問題の全体像が見えてくると思います」

湊の提案で、メンバーたちは立ち上がり、ホワイトボードの前に集まった。それぞれが付箋を見つめながら、どこに分類すべきか考えている。

「この『仕様変更が多い』と、この『要件定義が曖昧』は、同じグループじゃない？」

佐藤が指差しながら言う。

「そうだね。それから、『見積もりが外れる』も、要件定義の問題かもしれない」

別のメンバーが続ける。

「じゃあ、この辺りに『要件定義・仕様変更』っていうグループを作ろうか」

山田が付箋を動かしながら言う。

「『技術的負債で新機能が作りにくい』と、『コードが複雑すぎる』は、同じグループだね。それから、『テスト時間が足りない』も、品質の問題だから、ここに入るかもしれない」

「でも、『テスト時間が足りない』は、時間の問題でもあるよね」

高橋が付箋を一瞥してから疑問を投げかける。

「確かに。でも、根本的には品質保証の問題だと思う。時間が足りないのは結果であって、原因は別にあるかもしれない」

山田と高橋が議論を始めた。他のメンバーも、それぞれの意見を出し合いながら、付箋を動かしていく。

議論が深まる

「『残業が多い』と『時間が足りない』は、同じグループかな？」

「それと、『コードレビューする時間がない』も、時間の問題だよな」

「じゃあ、『時間・リソース不足』っていうグループを作ろう」

議論が進むにつれて、自然に5つのカテゴリが浮かび上がってきた。

「要件定義・仕様変更」

「技術的負債・品質」

「時間・リソース不足」

「コミュニケーション」

「評価・指標のズレ」

各カテゴリに分類された付箋を見て、メンバーたちは改めて問題の構造を理解していた。

飛鳥がホワイトボードを見つめながら口を開いた。

「こうして見ると、構造的な問題が多いですね。個人的な努力では解決できない、組織レベルの課題だと思います。例えば、『要件定義・仕様変更』の問題は、PM だけの問題でも、エンジニアだけの問題でもない。プロセス全体の問題です」

「そうですね。『技術的負債・品質』も、エンジニアだけの問題ではなく、QA 部やプロダクト部も関わってくる問題です」

高橋が頷いてから続ける。

「『時間・リソース不足』も、個人の努力では解決できない。組織全体で時間の使い方を見直す必要があると思います」

湊は5つのカテゴリを見つめながら、改めて問題の深刻さを実感していた。これらは、すべて関連し合っている。一つの問題が、別の問題を引き起こしている。だからこそ、チーム全体で解決していく必要があるのだと。

湊は、先日部長から聞いた話を思い出した。開発生産性を上げるには、開発スピードだけではなく、開発に関わる全ての業務を見直す必要がある、と。ミーティングの時間を本当に必要なものに絞る。運用で手作業にしていることをスク립トで自動化する。そうした業務プロセスの最適化も、チームで考えていくべきだ。湊はそれをメンバーに短く共有した。高橋が頷き、山田が「会議の見直しは、うちもやりたいところだな」と呟く。

形骸化した指標の発見

5つのカテゴリが整理された後、湊は「評価・指標のズレ」のグループを詳しく見てみることを提案した。

「このグループには、いくつか興味深い付箋があります。詳しく見てみませんか？」

メンバーたちは「評価・指標のズレ」のグループに集まった。そこには、5枚の付箋が貼られていた。

「ストーリーポイントが形骸化してる。見積もりを水増しする場合もあるし、簡単なタスクばかりを選びがちな自分もいる」

佐藤がその付箋を指差しながら言う。

「これ、僕が書いたんです。先月のスプリント計画で、同じようなタスクばかり選んで、ストーリーポイントを稼いだりしていました。難しいタスクはできなかったらどうしようというプレッシャーがあって。本当に必要な機能が後回しになってしまう」

「コード行数で評価されても……短く書くことが価値なのに、行数を増やすインセンティブが働く」

別のメンバーが続ける。

「以前、コード行数で評価されていた時期があったんです。それで、無駄なコードを書いて行数を増やす人が出てきました。リファクタリングしてコードを短くしたら、逆に読みにくくなったという評価が下がってしまった。スキル不足もあると思うんですが、それ以降リファクタリングする人が減りました」

「リリース頻度だけ見られても品質が……小さな変更を細かくリリースすれば数字は上がるけど、本当に価値があるのかわからない」

飛鳥が頷きながら言う。

「プロダクト部としても、リリース頻度を重視されることがあります。でも、小さな変更を細かくリリースしても、ユーザーには価値が伝わらない。むしろ、大きな機能を一度にリリースした方が、ユーザーには価値があることもあるんです」

湊はこれらの発言を聞きながら、数ヶ月前、田中部長から「開発生産性を上げてくれ」と言われた時のことを思い出していた。その時は、「どうやって数値化すればいいのか」と悩んでいた。でも、今は違う。単一の数値で開発活動を表そうとするとかえって歪む、という指摘が以前読んだどこかにあった。数値化することは手段であって、目的ではない。本質的な価値を創出することが目的なのだと理解できた。

「これらの指標って、本質的な問題解決につながってないですよね」

湊の言葉に、全員が深く頷く。

「そうそう、まさにそれ！」

「**指標を上げることが目的になっちゃってる**。本当に価値のあることをするよりも、数字を上げることの方が評価される」

「本当に必要なこと（リファクタリングとか、技術調査とか）が評価されない。だから、誰もやらなくなる」

山田がホワイトボードに目をやり、冷静に口を開いた。

「**グットハートの法則**って知ってる？ 『指標が目標になると、その指標は機能しなくなる』というものだ。指標を上げるための行動が起きる。でも、それは本質的な改善とは違うんだよね」

「例えば、ストーリーポイントを上げるために、簡単なタスクばかり選ぶ。コード行数を増やすために、無駄なコードを書く。リリース頻度を上げるために、小さな変更を細かくリリースする。これらは、すべて指標を上げるための行動であって、本質的な改善ではない」

「**キャベルの法則**も同じ。状態が数値を作るのであって、数値が状態を作るわけではない。数値を高めると強い組織になるのか、強い組織の状態が数値を高めるのか、その違いを理解しないと、指標が形骸化してしまう」

湊は深く頷く。数ヶ月前は数値化が目的だと思っていた。今、**その理解が覆った手応えを感じていた。**

「じゃあ、私たちは本当に何を大切にしたいんでしょうか？」

山田が少し間を置いて口を開いた。「ストーリーポイントを細かく合わせるより、何を達成したいかを揃える方が大事だと思うんだよね」

湊の問いかけに、会議室が再び静かになった。でも、今度は前向きな静けさだった。答えを探そうとする、そんな空気が漂っている。

解説 なぜ指標が形骸化するのか

⑤ ストーリーで描かれる「重圧」

「重圧から生み出される小手先の指標」が重圧になるのは、**課題が言語化される前に数値目標だけが先行してしまうためです。**湊のチームが直面したのも、この構造から生じています。

- **ワークショップのぎこちなさ** 最初の5分間は重い沈黙。付箋に「痛み」を書くと言われても遠慮がちで、当たり障りのない内容から始まる。チーム内のコミュニケーションの壁を感じている
- **形骸化した指標への気づき** 「ストーリーポイントが形骸化してる」「コード行数で評価されても…」「リリース頻度だけ見られても品質が…」とメンバーから声上がる。評価・指標のズレが一つのカテゴリとして浮かび上がる
- **本質からの乖離** これらの指標が本質的な問題解決につながっていないという共通認識が生まれる。指標を上げることが目的化し、本当に必要な改善が評価されないもどかしさ
- **重圧の正体** 重圧から生み出される「小手先の指標」に依存している現状。数値目標が先行し、開発活動の複雑さを単一の数値では表現できていない

この重圧の背景には、**指標が目標化し本質的な価値創出から乖離する構造**があります。形骸化した指標を特定し、本質的な問題に目を向ける必要性に直面

！ しているのです。

⑤ なぜ指標が形骸化するのか

湊のチームが発見した問題は、多くの開発組織が直面する典型的な課題です。

指標が形骸化する主な原因は以下の通りです。

- **グットハートの法則・キャベルの法則** ストーリーで山田が触れた通り、指標が目標になると本質から乖離し、状態と数値の因果を誤ると形骸化する。
- **指標の限界** 単一の数値では複雑な開発活動を表現できません。コード行数、コミット数、リリース頻度など、それぞれに限界があります。
- **ゲーミフィケーション** 指標を達成することが目的化し、本質的な価値創出を見失います。メトリクスを評価に直結させるとインセンティブが歪みゲーミングを誘発しやすいという問題意識が生まれます。SPACE（Satisfaction／Performance／Activity／Communication／Efficiency）の多次元で扱うべきだとし**活動量だけの増加は長時間労働や悪いシステムの力技で悪化しようと警告しています。**
- 合わせて、開発者の幸福と開発生産性に関する研究（グラツィオティンとファージャホルム（2019））では、開発者の不幸福が遅延や低品質のコード、プロセスからの逸脱、離職意向などと結びつきうることが論じられています。幸福を増やす介入より、不幸福要因を減らすほうが費用対効果が高い場合がある、とも述べられています。単一指標を追うことの限界と対をなすように、**数字に表れにくい不満や心理的負荷を扱う必要がある、**という示唆は、ここまでの指標批判と各職種の痛みを共有する対話と通じます

⑥ 見積りの精度よりゴールの共有

実装がボトルネックになっていたころは、プロダクトバックログの見積もりやスプリントに何をどれだけ入れるかの検討が重視されました。ウォーターフォールではPMの見積もりが計画の要でした。いずれも、作れる量が開発者数とスキルで決まり、プロダクト開発の速度を左右するという前提に立っています。

実装時間が相対的に短くなる前提では、細かい見積りを合わせるより、**何を達成したいか（ゴール）**を揃える方が重要になります。ストーリーポイントの

ゲーミングや見積り精度への過剰なこだわりは、本質的な価値やゴールの議論が足りていないことの裏返しです。この状況で最初にすべきことは、指標の微調整ではなく、チームとステークホルダーで「**どんなゴールを達成するか**」「**何をやらないか**」を明示的に合意することです。

⑤ 小手先の指標の問題点

形骸化した指標は、以下の問題を引き起こします。

- **マージPR数・タスク完了数の競い合い** レビューを薄くして数を稼ぐ、チケットを細かく分割して完了件数を水増しする、など活動量は増えても品質やユーザー価値が伴わない
- **変更リードタイムやサイクルタイムの単純目標化** 短ければよいと決めつけると、スコープを切り詰める、手戻りを別指標に逃がす、などで数値だけを整えたように見える
- **ストーリーポイントの形骸化** 見積もりを水増しする、簡単なタスクばかり選ぶ、難しい課題に誰も取り組まなくなる
- **リリース頻度での評価** 品質を犠牲にして小さな変更を細かくリリースする。本当に価値のある機能ではなく、数字を上げるためのリリースになる
- **AIコーディング支援の利用量や採用率** 量や採用率を増やすことが目的化し、設計・レビュー・セキュリティ確認が後回しになる
- **本質的な問題解決を妨げる** 本当に必要な改善（リファクタリング、技術調査、ドキュメント作成）が評価されず、誰も取り組まなくなる

⑤ 消える開発生産性と「次の一手」の設計

指標を改善しても、組織の成果に届かないことがあります。その理由の一つが**消える開発生産性（Vanishing Productivity）**です。効率化によって生まれた時間や余力が、組織の慣性に吸収され、最終的な価値創造につながらない現象を指します。例えば、空いた時間が価値の低い仕事や形式的な会議に使われたり、チームレベルではタスク完了数・マージPR数が大きく増えているのに、組織レベルでは全社スループット・品質指標に有意な改善が見られなかったりします。

Faros AI が2025年7月に公表した *The AI Productivity Paradox (AI Engineering Impact Report)* では、タスク管理・IDE・静的解析・CI/CD・バージョン管理・

インシデント管理・人事メタデータなどのデータを複数社にまたがる1,255チーム・1万人超の開発者を対象に分析した結果が報告されています。AI利用度の高いチームではタスク完了数が約21%増、マージPR数が約98%増だった一方、組織レベルでは全社スループット・DORA指標・品質KPIに**有意な相関がなく、PRレビュー時間が約91%増加、バグ発生率が約9%上昇していました**。下流のボトルネックが価値を吸収している例です。個人の稼働を高めたりタスク完了数を増やしたりしても、価値が顧客に届くまでの流れが詰まっていれば組織の成果には結びつきません。指標改善が組織成果に届かない理由の一つは、「どれだけ動いたか」ではなく「どれだけ早く届いたか」を見ていないことにあるのです。

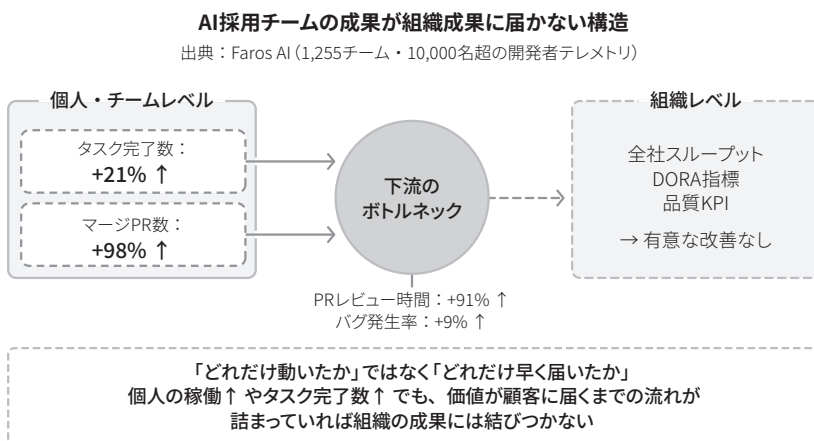


図4.1：個人・チームレベル（タスク完了+21%、マージPR+98%）、下流ボトルネック（PRレビュー時間+91%、バグ発生率+9%）、組織レベル（全社スループット・DORA・品質KPIに有意な改善なし）の関係を示す概念図

重要なのは、「次の一手」を設計しておくことです。効率化で空いた時間を何に使うか（やめる仕事を決める、成果指標を忙しさから価値へ変える、学習へ再投資する）を事前に決めていないと、組織の慣性が余力を吸い取ります。湊のチームが「本質的な価値とは何か」を問い、痛みを共有したうえで指標を見直すのは、この「次の一手」を設計するプロセスそのものです。

⑤ 問題を特定するための手法

そうした形骸化した指標を発見し、本質的な問題を特定するためには、以下

の手法が有効です。

- **指標がゲーム化されていないかチェック** 指標を上げるための行動が起きていないか、本質的な改善と乖離していないかを確認
- **指標と実際の価値創出の因果関係を確認** 指標が上がっても、実際の価値創出（ユーザー満足度、ビジネス成果）につながっているかを検証
- **多面的な指標の組み合わせ** 単一指標に依存せず、複数の指標を組み合わせで評価する
- **定期的な指標の見直し** 四半期ごとに指標の妥当性をレビューし、必要に応じて変更する

詳細な手法については、章末の「手法4 脱・形骸化指標リスト」を参照してください。

4-2

それぞれの立場から見た 「痛み」を共有し理解する対話

ストーリー

痛みの共有と相互理解

形骸化した指標について議論した後、湊は各職種の痛みをもっと詳しく聞いてみることにした。

「それぞれの立場で、どんな痛みを感じているか、もう少し詳しく聞かせてください。エンジニア、QA、PM、それぞれの視点で、問題をどう見ているか知りたいんです」

ワークショップが進むにつれて、各職種の痛みが明確になってきた。ホワイト

ボードの付箋が、それぞれの言葉で色を増していく。

高橋が深く息を吸い込んでから、話し始める。

「わたしの感覚だとバグが多いと結局後で工数が10倍かかる。要件定義段階が1倍、実装段階で10倍、リリース後は30-100倍。品質も開発生産性のうちだと思うんですけどね。テスト時間を削ってリリースを早めても、結局後でバグ対応に時間がかかって、トータルでは時間がかかってしまう」

高橋の言葉に、エンジニアのメンバーたちも頷く。誰もが同じ経験をしていた。

山田が腕を組み、少し考えてから口を開いた。

「高橋さんの言う通り、品質も開発生産性のうちだと思います。ただ、品質と言っても多面的な側面があります。この前見たレポートの中でISO/IEC 25010という国際規格があり、ソフトウェア品質を8つの特性に分類できるらしいです。機能適合性、性能効率性、信頼性、保守性など、それぞれが重要な側面です。今のチームの課題を整理する際も、このような多面的な視点で品質を捉える必要があるかもしれませんね」

山田の言葉に、高橋も深く頷く。

「プロダクト部として、ユーザー価値を届けたいんです。でも、開発速度が遅いと感ずることもあって……技術的負債の重要性は理解したいんですが、ビジネス的なプレッシャーもあって」

飛鳥が頷きながら続ける。

「実は私も悩んでる。短期的な売上は大事だけど、長期的な持続可能性も大事。そのバランスが難しいんです。経営陣からは『もっと早く機能を出せ』と言われる。でも、開発チームからは『技術的負債を返済する時間が必要』と言われる。どちらも正しいと思うんですが、どうバランスを取ればいいのかかわからない」

「それから、ユーザーからの要望も多いんです。『この機能を追加してほしい』『あ

の機能を改善してほしい』。それらを全部開発チームに伝えたと、『優先順位をつけてほしい』と言われる。でも、**ビジネス的には全部重要に見える**。その優先順位をつけるのが難しいんです」

飛鳥の言葉に、山田が反応した。

「技術的な視点から言うと、技術的負債を無視して新機能を追加し続けると、開発速度はどんどん低下する」

「でも、それをどうビジネス側に説明すればいいのか。技術用語で説明しても理解してもらえない。『技術的負債』って言葉自体が、ビジネス側には伝わりにくい。『コードが複雑になっている』と言っても、それがどうビジネスに影響するのか、説明するのが難しい」

山田の言葉に、飛鳥が頷く。

「確かに技術的な話をされても、私には理解できないことが多いです。でも、それがビジネスにどう影響するのか、もっとわかりやすく説明してもらえれば、理解できると思います」

湊はメンバーの話を聞きながら、数週間前、各職種にヒアリングした時のことを思い出していた。それぞれの立場で「開發生産性」の定義が違う。でも、それは対立の原因ではなく、相互理解の出発点なのだと気づいた。

各職種の痛みが交差する

「みんな、それぞれの立場で正しいことを言ってるんですよね。エンジニアは開発プロセスと技術的品質を重視する。QAは品質保証を重視する。PMはユーザー価値を重視する。でも、それがバラバラだから問題が起きる」

湊は少し間を置いてから続ける。

「でも、今日のワークショップで気づいたことがあります。それぞれの立場は違う

けど、感じている痛みは共通している。『時間が足りない』『開発速度を早くしたい』『品質を保ちたい』『ユーザーに価値を届けたい』。これらは、すべての職種が感じていることです」

「じゃあ、どうすればいいんでしょうか？」

佐藤が不安そうに聞く。

「まずは、お互いの痛みを理解することからだと思います。エンジニアの痛み、QAの痛み、PMの痛み。それぞれを理解して、共通の課題を見つける。そして、その共通の課題を、チーム全体で解決していく」

湊はホワイトボードの5つのカテゴリを指差しながら言う。

「これが、私たちの共通の課題です。『要件定義・仕様変更』『技術的負債・品質』『時間・リソース不足』『コミュニケーション』『評価・指標のズレ』。これらは、エンジニアだけの問題でも、QAだけの問題でも、PMだけの問題でもない。チーム全体の問題です。だからこそ、チーム全体で解決していく必要があると思います」

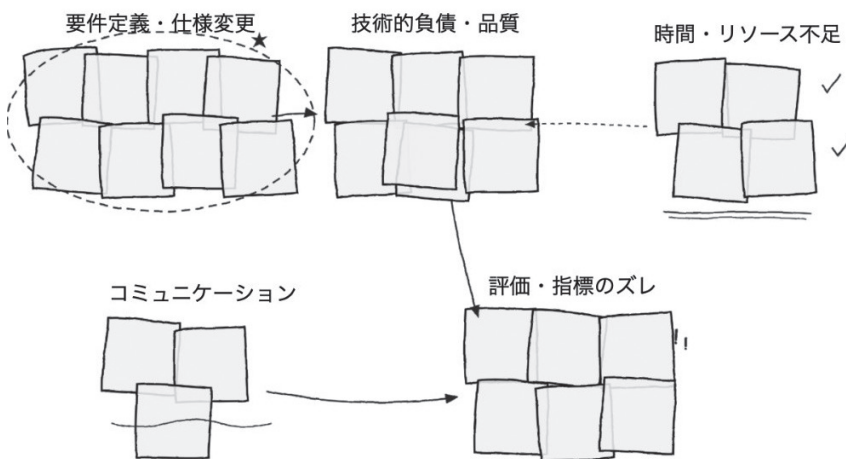


図 4.2：5カテゴリの無地付箋・ラベル・矢印による関係の概念図（手描き風）

湊は各カテゴリを順番に指差しながら説明した。

「『要件定義・仕様変更』の問題は、PMだけの問題でも、エンジニアだけの問題でもない。プロセス全体の問題です。要件定義の段階で、PM、エンジニア、QAと一緒にレビューすれば、仕様変更を減らせるかもしれません」

「『技術的負債・品質』の問題も、エンジニアだけの問題ではありません。技術的負債が蓄積すると、開発速度が低下し、結果としてユーザーに価値を届けるのが遅くなる。それは、PMにとっても、QAにとっても、問題です」

「『時間・リソース不足』も、個人の努力では解決できません。組織全体で時間の使い方を見直す必要があります。コードレビューの時間を確保する、テスト時間を確保する、リファクタリングの時間を確保する。これらは、すべて組織レベルの意思決定が必要です」

「『コミュニケーション』の問題も、チーム全体の問題です。エンジニアとPM、エンジニアとQA、それぞれの間で情報が共有されていない。だから、認識のずれが起きる。定期的な対話の場を作ることで、改善できるかもしれません」

「そして、『評価・指標のズレ』の問題。これは、私たちが今日発見した問題です。形骸化した指標に依存せず、本質的な価値を創出することを目指す」

湊の説明に、メンバーたちは真剣に聞き入っていた。それぞれが自分の問題だと思っていたことが、共通の課題として見え、チーム全体のものだとして理解していた。

「じゃあ、これからどうすればいいんでしょうか？」

佐藤が質問する。

「まずは、これらの課題を一つずつ解決していくことだと思います。でも、全部を一度に解決するのは難しい。優先順位をつけて、小さなステップから始める。そして、定期的に振り返って、改善していく」

湊はメンバーたちを見回しながら続ける。

「今日は、痛みを共有して、共通の課題を見つけることができました。これが第一歩です。次は、この課題をどう解決していくか、一緒に考えていきましょう」

全員が頷く。それぞれが個別だと思っていた問題が、共通の課題として一つになった。そして、それをチーム全体で解決していく決意が、全員の間に生まれていた。

解説 なぜ痛みの共有をするのか

◇ ストーリーで描かれる「重圧」

痛みが表に出る前に重くのしかかるのが、「痛みを共有するまでに越える壁」です。湊のチームが経験した重圧も、同じ構造から生じています。

- **本音が出るまでの壁** 高橋の「テスト時間が全然足りません」という一言をきっかけに、山田や佐藤らも本音を話し始める。それまで表に出ていなかった痛みが付箋で可視化され、一気に増えていく
- **各職種の痛みの可視化** 技術的負債で身動きが取れない、見積もりが外れる理由がわからない、コードレビューの時間がない、モジュール間の複雑さ、属人化した設計。要件定義・仕様変更、技術的負債・品質、時間・リソース不足、コミュニケーション、評価・指標のズレの5カテゴリに整理される
- **「みんな同じことで悩んでた」** バラバラに見えた問題が構造的な課題として共有される。一人で抱え込んでいた問題がチーム全体の課題になり、対立ではなく協働の土台ができる
- **痛みの共有がもたらす転換** 本音を言えない環境から、痛みを出し合うことで問題の全体像が見え、共通の課題を発見する段階に進んでいる

この重圧の背景には、**情報の非対称性や心理的安全性の欠如で痛みが表面化せず、共有することで初めて共通課題と協働の基盤が生まれる構造**があります。痛みの共有という第一歩を踏み出した段階にあるのです。

開発生産性に関する研究では、活動量だけでなく**満足度・ウェルビーイング**（SPACEフレームワークの「S」）や、フィードバックループ・認知負荷・フロー状態（DevExの3次元）と結びつけて考える必要があるとされています。仕事満足度と知覚的生产性には双方向の因果があり、自己評価の開発生産性と強く相関するのはツールより**仕事への熱意・ピアサポート・有益なフィードバック**といった非技術的要因であるとする知見があります。痛みを出し合い、互いの立場を理解し、心理的安全性を高めることは、数値では測りにくいですが持続可能な開発生産性の基盤となります。

痛みを共有することで、以下の効果が得られます。

- **問題の全体像が見える** 各職種が個別に感じている問題が、実は構造的な課題であることがわかる
- **相互理解が深まる** エンジニア、QA、PMそれぞれの立場での苦労を理解できる
- **共通の課題を発見** バラバラに見える問題が、実は同じ根本原因から生まれていることがわかる
- **協働の基盤ができる** お互いの痛みを理解することで、対立ではなく協働の関係構築

② ソフトウェア品質特性 (ISO/IEC 25010:2011) による品質の多面的理解

ワークショップで山田も言及していましたが、ソフトウェア開発において「品質」に関する言語化が難しいとされています。

銀の弾丸はありませんが、一つのヒントになるものとして**ISO/IEC 25010**という規格があります。ソフトウェア品質を体系的に理解するための国際規格です。品質を単一の指標で測らず、**機能適合性・性能効率性・互換性・使用性・信頼性・セキュリティ・保守性・移植性**の8つの特性に分類して多面的に評価できます。



図 4.3：ソフトウェア製品品質モデル ISO/IEC 25010:2011（8つの品質特性と31の品質副特性、利用時の品質）

品質特性を活用した課題整理

「技術的負債・品質」カテゴリの課題を、この枠組みで整理すると次のようになります。

- **保守性** コードの複雑度が高く修正しにくい、テストが不十分でテストしにくい
- **信頼性** バグが多く障害が発生しやすい、回復が遅い
- **性能効率性** ビルド時間が長い、レスポンスが遅い
- **セキュリティ** ライブラリのバージョンが古く、パッチが適用されていない

品質特性で整理すると課題の優先順位をつけやすくなります。各特性は開発生産性に直結し、保守性・信頼性が低いと開発速度が落ち、バグ対応に時間を取られます。品質特性への投資は、短期的には開発速度を抑えるように見えても、長期的には開発生産性向上につながります。

4-3

チーム全体で合意できる 本質的な価値の発見

ストーリー

本質的な価値の模索

痛みを共有し、共通の課題を発見した後、湊は次のステップを提案した。

「じゃあ、私たちは本当は何を大切にしたいんだろう？ 形骸化した指標ではなく、本質的な価値について考えてみませんか？」

ブレインストーミングの熱気

ブレインストーミングが始まった。会議室の空気が、少し前向きに動き出す。

「ユーザーに価値を届けること。機能の数ではなく、ユーザーの問題を解決すること」

飛鳥が最初に口を開いた。

「持続可能な開発速度。短期的なスピードではなく、長期的に維持できる速度」

山田が少し考えてから続ける。

「チーム全員が成長できる環境。学習機会、心理的安全性、相互支援」

高橋が加わる。

「品質を保ちながらスピードも出す。トレードオフではなく、両立を目指す」

佐藤も小さく言う。

「コードレビューする時間がある。急がず、丁寧に開発できる環境」

他のメンバーも次々と意見を出した。

「技術的負債を適切に管理する。後回しにせず、計画的に返済する」

「要件定義に十分な時間をかける。仕様変更を最小限に抑える」

「コミュニケーションが活発。情報が共有され、認識のずれが起きない」

山田が少し間を置き、別の観点を追加した。

「事業への貢献が実証されているシステムこそ、ドメイン駆動設計で取り組む価値が最も高い。このシステムは、すでにユーザーに価値を提供している。だからこそ、長期的な持続可能性を確保するために、ドメインモデルを明確にし、イベントストーミングで設計を整理する必要がある。積み重なった技術的負債と膨れ上がった設計の乱雑さを刷新することで、将来の開発速度を大幅に向上させられる」

飛鳥が手を挙げた。

「進捗はチケットで報告していますが、それだと消化率ばかりが話題になります。本当に大事なのは、短い間隔で動くソフトウェアを見せることではないでしょうか」

佐藤が頷いて口を挟んだ。

「モックや仕様書だと反応が薄かったのに、実物を見せたらフィードバックが一気に増えたことがあります」

湊はその言葉をホワイトボードに追記した。定期的に動作するソフトウェアをステークホルダーに見せる。ホワイトボードには、本質的な価値が並んでいた。

湊はそれらを見つめながら言う。

「これが私たちの『**本当の開発生産性**』なのかもしれませんね。数値ではなく、価値創出と持続可能性。それが、私たちが目指すべきものだと思います」

どれだけ作れるかで振り回されるより、どんなゴールを達成するかで判断したい。湊はホワイトボードの言葉を見ながら、そう思った。

アクションプランの策定

本質的な価値が明確になった後、湊は次のステップを提案した。

「じゃあ、これらの価値を実現するために、具体的に何をすればいいのでしょうか？ 各カテゴリに対する改善アクションを考えてみませんか？」

メンバーたちは、5つのカテゴリを見ながら改善策を考え始めた。

「要件定義・仕様変更」について、飛鳥がそのグループを指差してから提案した。

「事前レビュー会を導入してはどうでしょうか？ PM、エンジニア、QAの3者で、実装前に要件をレビューする時間を取りたいです」

「技術的負債・品質」について、山田が提案した。

「週に1日、リファクタリングやライブラリのアップデートをする時間を確保する。開発時間の20%を技術的負債の返済に充てる。初めは開発スピードがもしかしたら落ちるかもしれないけど、2か月後にはその前よりも開発スピードが上がっているかを確認したい」

「品質保証の仕組み作り」について、高橋が提案した。

「自動テストを導入する。継続的インテグレーション／継続的デリバリー（CI/CD）パイプラインで自動テストを実行して、品質を担保できるような仕組みを

作りたい」

「チーム内コミュニケーション改善」について、湊が提案した。

「朝会を導入する。毎朝 15 分、チーム全員で進捗を共有する。デイリースタンドアップです」

4つの重点アクションが決まった。加えて、週次で動作するソフトウェアをPMや関係者に見せる機会を設けることで合意した。

1. 要件定義プロセスの改善（飛鳥・プロダクト部担当）
2. 技術的負債返済時間の確保（テックリードの山田担当）
3. 品質保証の仕組み作り（QA 部の高橋担当）
4. チーム内コミュニケーション改善（湊リーダー担当）

それぞれ2週間後に進捗報告をすることになった。

「よし、やってみよう！」

全員が前向きに頷く。一人で抱え込んでいた問題が、チーム全体の課題として共有され、解決に向けて動き始めた。

アクションプランの実行と成果

2週間後、フォローアップミーティングが開催された。前回と同じ大会議室に、同じメンバーが集まっている。各担当者から、試行錯誤を経て得られた成果が報告された。

飛鳥が最初に報告した。

「要件定義の事前レビュー会を導入しました。最初はエンジニアとQAのスケジュールが合わなくて大変でしたが、続けるうちにコツがつかめてきました。仕様変更が週に3-5回だったのが、週に1-2回に減りました」

続けて、山田が同じ調子で続ける。

「週1回、リファクタリングの時間を確保しました。毎週木曜日の午後にしました。最初の週は緊急のバグ対応が入って時間が取れませんでした、2週目からは守れるようになりました。まだ大きな成果は出ていませんが、徐々に手応えを感じつつあります」

高橋も報告した。

「一旦、SaaSを無料トライアルを契約して簡単な自動テストの導入を開始しました。最初はテストが失敗してデプロイが止まってしまいましたが、エンジニアのメンバーが協力して修正できました。まだE2Eテスト（End-to-End）のカバレッジは低いですが、徐々に増やしていく予定です」

湊が最後に報告した。

「朝会を導入しました。毎朝15分、チーム全員で進捗を共有します。最初は時間がかかっていましたが、タイマーを使い、『昨日やったこと、今日やること、困っていること』の3つに絞ることで、15分で終わるようになりました。情報の透明性が高まり、認識のずれが減りつつあります」

飛鳥が補足した。

「動くものを見せる機会を設けたら、話が『いつリリース？』から『ここはこう変えたい』に変わってきました。最近はちょこちょこ田中部長も顔を出すようになったんですよ……！」

小さいながらも確実な改善を実感していた。

「少しずつだけど、変わってきてますね」

湊はメンバーたちを見ながら言う。

「一人で抱え込んでいた時は、何も変えられなかった。でも、チーム全体で考えれ

ば、きっと道は開ける」

山田が微笑み、少し間を置いてから口を開いた。

「湊くん、君が種を蒔いた木が、少しずつ育ってきてるね」

解説 本質的な価値とは何か

⑤ ストーリーで描かれる「重圧」

「本当に何を大切にしたいか」と問い詰められた経験があるなら、それは**本質的な価値**に目を向け始めた段階で現れる構造的な課題の表れかもしれません。湊のチームもそこに立っています。

- **「本当に何を大切にしたいか」** 痛みのグルーピングと形骸化した指標への気づきの先に、チームで本質的な価値を模索する問いが立つ。ユーザーに価値を届けること、持続可能な開発速度、チーム全員が成長できる環境、品質を保ちながらスピードも出す
- **数値では測れない価値** ストーリーポイントやコード行数では表せない「本当の開発生産性」を言語化する。事業に貢献しているシステムだからこそドメインモデルを明確にし、技術的負債と設計の乱雑さを刷新する必要性が共有される
- **アクションへの落とし込み** 4つの重点アクション（要件定義プロセスの改善、技術的負債返済時間の確保、品質保証の仕組み作り、チーム内コミュニケーション改善）を担当者と期限で決め、週次で動作するソフトウェアを関係者に見せる機会の設定にも合意し、全員が「よし、やってみよう」と前向きになる
- **最初の成果** 2週間後のフォローアップで、事前レビュー会の導入、週1回のリファクタリング時間確保、自動テスト導入の開始、朝会での情報共有改善に加え、動くものを見せる機会を設けたことで会話が「いつリリース？」から「ここはこう変えたい」へ変わり見に来る人が増えたことなど、小さくとも確かな改善が報告される

この重圧の背景には、**形骸化した指標に依存せず、数値では測れない本質的な価値をチームで合意し、アクションプランに落とすことで継続的改善が動き始める構造**があります。転換を経て、協働による改善の第一歩を踏み出しているのです。

⑤ 本質的な価値とは何か

湊のチームが発見した本質的な価値は、多くの開発チームが目指すべき目標です。

本質的な価値とは、数値では測れない、しかし開発チームが目指すべき価値です。

- **ユーザーに価値を届けること** 機能の数ではなく、ユーザーの問題を解決すること。ユーザー満足度、ビジネス成果につながる価値
- **持続可能な開発速度** 短期的なスピードではなく、長期的に維持できる速度。技術的負債を適切に管理し、チームの健全性を保つ
- **チーム全員が成長できる環境** 学習機会、心理的安全性、相互支援。チームメンバーが成長することで、長期的な開発生産性が向上する
- **品質を保ちながらスピードも出す** トレードオフではなく、両立を目指す。自動テスト、CI/CD、コードレビューなど、品質を担保する仕組みを整える
- **定期的に動作するソフトウェアをステークホルダーに見せる** 進捗をチケット消化や報告会だけで伝えるのではなく、短い間隔で動くものを見せることで、フィードバックと信頼が積み上がる

⑥ 定期的に動作するソフトウェアを披露する意義

進捗をチケット消化や報告会だけで伝えるやり方と、短い間隔で動作するソフトウェアを披露するやり方では、得られるフィードバックと信頼が違います。

披露することを習慣にすると、いくつかの改善が連鎖します。(1) 一気通貫で見て評価できるものを作る必要に迫られるため、部品やモックだけでは済まなくなる。(2) 短い期間で多様な仕事をこなす必要から、チーム内のスキルが揃い、オーバーラップが進む。(3) 見る人の関心に合わせたテーマや目玉が必要になり、チームが事前に準備する習慣ができる。(4) 関心のあるものを毎回見せていると信頼が積み上がり、実験もしやすくなる。

なぜ定期披露が重要か。プロダクトの価値はチームの外側で決まります。チーム内に留めている限りアイデアは検証されず、時間とコストをかけた後の失敗リスクが高まります。定期的に披露することは、そのリスクへの対処でもあります。

⑤ チーム全体での合意形成の重要性

チーム全体で本質的な価値に合意することで、以下の効果が得られます

- **共通の目標を持つ** チームの方向性が明確になり、個々の判断が統一される
- **各職種の視点を統合** エンジニア、QA、PMそれぞれの視点を統合することで、バランスの取れた判断ができる
- **協働の基盤を構築** 共通の価値観を持つことで、対立ではなく協働の関係を築ける
- **継続的な改善** アクションプランを実行し、定期的に振り返ることで、継続的な改善が可能になる

⑥ アクションプランの実行と継続的改善

アクションプランを実行し、継続的に改善するためには、以下のポイントを押さえます。

- **小さなステップから始める** 大きな変化ではなく、小さな改善から始める。2週間、1ヶ月という短いサイクルで進捗を確認する
- **定期的な振り返り** 四半期ごとに、アクションプランの効果を振り返り、必要に応じて調整する
- **成果の可視化** 小さな成果でも可視化し、チーム全体で共有する。モチベーションの維持につながる
- **継続的な改善** 一度の改善で終わらず、継続的に改善を続ける。開発生産性の向上は、継続的な取り組みによって実現される

詳細な手法については、章末の「手法4 脱・形骸化指標リスト」を参照してください。

手法4：脱・形骸化指標リスト

形骸化した指標のチェックリスト

以下の項目をチェックしてください。該当する項目が多いほど、指標が形骸化している可能性が高いです。

⚙️ 指標のゲーム化

- 指標を上げるための行動が起きている（例）コードを細かく分割してコミット数を増やす
- 簡単なタスクばかり選ぶ人がある
- 見積もりを水増しする人がある
- 本当に必要な仕事（リファクタリング、技術調査）が評価されない

⚙️ 指標と実際の価値創出の乖離

- 指標が上がっても、ユーザー満足度が上がらない
- 指標が上がっても、ビジネス成果につながらない
- 指標が上がっても、チームのモチベーションが上がらない
- 指標が上がっても、品質が向上しない

⚙️ 単一指標への依存

- 1つの指標（例えばコード行数、コミット数、リリース頻度）だけで評価している
- 指標の妥当性を定期的に見直していない
- 指標の意味をチーム全体で理解していない
- 指標が改善のツールではなく、評価のためだけに使われている

判定

- 0-2個該当 良好な状態。指標が適切に機能している
- 3-5個該当 注意が必要。指標の見直しを検討する
- 6-8個該当 危険な状態。指標が形骸化している可能性が高い

- 9-12個該当 深刻な状態。指標が逆効果になっている

本質的な価値を測る指標の選び方

形骸化した指標ではなく、本質的な価値を測る指標を選ぶためには、以下のポイントを考慮してください。

⚙️ アウトカム指標を重視する

- **アウトプットではなく、アウトカムを測定** コード行数、コミット数ではなく、ユーザー満足度、ビジネス成果を測定する
- **複数の指標を組み合わせる** 単一指標に依存せず、複数の指標を組み合わせで評価する
- **定性的な評価も併用** 数値では測れない価値（チームの健全性、心理的安全性）も評価する

⚙️ DORA Metrics の活用

DORA Metrics（Four Keys）は、形骸化した指標とは対照的に、本質的な価値創出につながる指標として機能します。以下の4つの指標を組み合わせることで、バランスの取れた評価が可能になります。

Four Keysの詳細

1. デプロイ頻度（Deployment Frequency）

- **測定方法** 本番環境へのリリース回数を期間で割る（例）週1回、月1回
- **意味** 組織がどれだけ迅速に価値を届けられるかを示す
- **注意点** 頻度だけを追うのではなく、価値のある変更かどうかを確認する

2. 変更のリードタイム（Lead Time for Changes）

- **測定方法** コードコミットから本番デプロイまでの時間を測定
- **意味** アイデアから価値提供までの時間を示す
- **注意点** リードタイムが短いほど、市場への対応速度が上がる

3. 変更失敗率（Change Failure Rate）

- **測定方法** 本番デプロイ後に障害が発生した回数を全デプロイ回数で割る
- **意味** デリバリーの品質を示す
- **注意点** 失敗率が低いほど、安定したデリバリーができています

4. サービス復元時間（Time to Restore Service）

- **測定方法** 本番障害の発生から復旧までの時間を測定
- **意味** 障害発生時の対応能力を示す
- **注意点** 復旧時間が短いほど、ユーザーへの影響を最小化できる

DORA Metricsの活用方法

- **チーム単位で測定** 個人評価ではなく、チーム全体の成果として測定する
- **定期的なレビュー** 四半期ごとに指標を確認し、改善の方向性を議論する
- **DORA Core Modelとの連動** 継続的デリバリー、バージョン管理などのプラクティスと連動させて改善する
- **ビジネス指標との関連付け** DORA Metricsの改善が、顧客満足度や売上などのビジネス指標にどう影響するかを追跡する

これらの指標を組み合わせることで、速度と安定性の両立を実現し、本質的な価値創出につながる評価が可能になります。

出典 DORA（DevOps Research and Assessment）レポート、
『Accelerate: The Science of Lean Software and DevOps』（ジーン・キム、
ジェズ・ハンプル、ニコール・フォースグレン著）

⚙️ チーム単位で測定する

- **個人評価には使わない** 指標を個人の評価に使うと、ゲーム化が起きやすい
- **チーム単位で測定** チーム全体の成果として測定し、改善のツールとして活用する
- **定期的に見直す** 四半期ごとに指標の妥当性をレビューし、必要に応じて変更する

指標の定期見直しプロセス

指標を形骸化させないためには、定期的な見直しが必要です。

⚙️ 四半期ごとのレビュー

1. **指標の妥当性を確認** 指標が本質的な価値創出につながっているかを確認
2. **ゲーム化のチェック** 指標を上げるための行動が起きていないかを確認
3. **指標の変更・追加** 必要に応じて指標を変更したり、新しい指標を追加したりする
4. **チーム全体での合意** 指標の変更について、チーム全体で合意を形成する

⚙️ 指標の見直しチェックリスト

- 指標が本質的な価値創出につながっているか
- 指標がゲーム化されていないか
- 指標と実際の価値創出の因果関係が明確か
- チーム全体が指標の意味を理解しているか
- 指標が改善のツールとして機能しているか

これらのチェックリストを定期的に確認することで、指標の形骸化を防ぐことができます。

章 の ま と め

本章では、湊のチームがワークショップを通じて、形骸化した指標の問題に気づき、本質的な価値を発見する過程を描きました。

⚙️ 主な学び

1. **指標の形骸化** グットハートの法則、キャベルの法則により、指標が目標になると本質を見失う
2. **痛みの共有** 各職種の痛みを共有することで、共通の課題を発見できる
3. **本質的な価値** 数値ではなく、価値創出と持続可能性が、本当の開発生産性です
4. **チーム全体での問題解決** 一人で抱え込むのではなく、チーム全体で問題を共

有し、解決に取り組む

🌀 次章への橋渡し

本章でチーム全体での問題解決を描いた。本章で決めた四つの重点アクションの実行と、その約1ヶ月後の様子が次章で描かれる。次章では、AIとの協働による持続可能な開発へと進む。

参考文献

本章で指標・痛みの共有・本質的価値を論じる際に参照した文献を、本文での言及順に挙げる。

★著者様：挙げます？

- **Petersen, K.** (2011). "Measuring and predicting software productivity: A systematic map and review." *Information and Software Technology*, 53(4), 317–343. doi:10.1016/j.infsof.2010.12.001. 一次研究38件を整理し、単純な比率指標にリスクがあること、データ包絡分析 (DEA) など参照比較に基づく枠組みが単純比率より頑健な比較の手がかりになりうることを示されている。本章の「指標の形骸化」の根拠。
- **DORA (DevOps Research and Assessment)** . DORA metrics guide. [<https://dora.dev/guides/dora-metrics/>] (<https://dora.dev/guides/dora-metrics/>) および継続的レポート。Four Keys (デプロイ頻度・変更リードタイム・変更失敗率・復旧時間)。速度と安定性の両立が統計的に示されている。『Accelerate: The Science of Lean Software and DevOps』(Gene Kim, Jez Humble, Nicole Forsgren 著) にまとめられている。
- **Forsgren, N. et al.** (2021). "The SPACE of Developer Productivity." 開発生産性を Satisfaction / Performance / Activity / Communication / Efficiency の多次元で扱い、活動量だけの増加は長時間労働や悪いシステムの力技で悪化しようと警告している。本章の痛みの共有・本質的価値の多面性の参照。
- **Graziotin, D., Fagerholm, F.** (2019). "Happiness and the Productivity of Software Engineers." In: Sadowski, C., Zimmermann, T. (eds.) *Rethinking Productivity in Software Engineering*. Apress (オープンアクセス)。不幸福と遅延・品質・離職意向などの関係、不幸福要因の除去の費用対効果が論じられている。本章の痛みの共有・指標の形骸化の根拠。
- **Faros AI.** (2025). *The AI Productivity Paradox (AI Engineering Impact Report, July 2025)* . タスク管理・IDE・静的解析・CI/CD・バージョン管理・インシデント管理・人事メタデータ等からテレメトリを集約し、1,255チーム・10,000名超の開発者を分析。ブログ要約 [<https://www.faros.ai/blog/ai-software-engineering>] (<https://www.faros.ai/blog/ai-software-engineering>)。本章の消える開発生産性・下流ボトルネックの根拠。
- **ISO/IEC 25010:2011**. システム及びソフトウェア工学—システム及びソフトウェア品質要求及び評価 (SQuaRE) —システム及びソフトウェア品質モデル。ソフトウェア品質の8つの特性 (機能適合性、性能効率性、互換性、使用性、信頼性、セキュリティ、保守性、移植性) を定義する国際規格。本章で山田が言及した品質の多面的理解の根拠。
- **吉羽龍太郎 (ryuzee)** . 「短い間隔で動作するソフトウェアを見せようとするすべてが改善される」. Ryuzee.com, 2024/08/26. <https://www.ryuzee.com/contents/blog/14593> 本章の「定期的に動作するソフトウェアを披露する意義」の参照。

